

Sistema de Monitoreo de Cámaras Frigoríficas

Arquitectura completa: Arduino + DHT11 → Backend API → Dashboard Web

Este documento describe la arquitectura de software para monitorear temperaturas en cámaras frigoríficas de conservación de fruta, garantizando el rango óptimo de **0 °C a 10 °C**. Incluye tres implementaciones de backend: Node.js, Python y PHP.

1. Visión general del sistema

El sistema captura datos físicos desde sensores DHT11 conectados a placas Arduino via USB/Serial, los procesa en un backend API y los presenta en una plataforma web con visualización en tiempo real, historial de temperaturas y soporte para múltiples cámaras frigoríficas simultáneas.

Flujo de datos entre capas:

Arduino + DHT11 Captura física	Script Serial Lector USB → HTTP	Backend API Validación + DB	Dashboard Web Visualización
--	---	---------------------------------------	---------------------------------------

■ Rango crítico: la temperatura debe mantenerse entre 0 °C y 10 °C. El backend emite una alerta inmediata si se detecta un valor fuera de este rango.

2. Capa 1 — Hardware (Arduino + DHT11)

Cada cámara frigorífica tiene un Arduino con un sensor DHT11 que mide temperatura y humedad. El Arduino envía los datos por puerto USB/Serial hacia la PC en formato de texto simple.

Sketch Arduino (código):

```
#include <DHT.h>
#define DHTPIN 2
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(9600);
  dht.begin();
}

void loop() {
```

```

float temp = dht.readTemperature();
float hum = dht.readHumidity();
// Formato: CAMERA_ID,TEMP,HUM
Serial.println("1," + String(temp) + "," + String(hum));
delay(5000); // cada 5 segundos
}

```

3. Capa 2 — Script lector serial (Python)

Este script corre en la PC, lee el puerto USB/Serial y reenvía los datos al backend mediante HTTP POST. Es independiente del backend elegido.

Instalación:

```
pip install pyserial requests
```

Código (serial_reader.py):

```

import serial, requests, time

PORT = 'COM3' # Windows: COM3 / Linux: /dev/ttyUSB0
BAUD = 9600
API_URL = 'http://localhost:3000/api/readings'

ser = serial.Serial(PORT, BAUD, timeout=2)

while True:
    line = ser.readline().decode('utf-8').strip()
    if line:
        cam_id, temp, hum = line.split(',')
        requests.post(API_URL, json={
            'camera_id': int(cam_id),
            'temperature': float(temp),
            'humidity': float(hum)
        })
    time.sleep(1)

```

4. Capa 3 — Backend API (tres implementaciones)

Los tres backends exponen exactamente los mismos endpoints REST. La diferencia es el lenguaje y las dependencias.

Comparativa de stacks:

	Node.js + Express	Python + FastAPI	PHP + MySQL
Lenguaje	JavaScript	Python	PHP
DB recomendada	SQLite / Postgres	SQLite / Postgres	MySQL / MariaDB
Tiempo real	Socket.io	WebSocket nativo	SSE / polling

	Node.js + Express	Python + FastAPI	PHP + MySQL
Doc. automática	No	Si (/docs)	No
Hosting simple	Node en VPS	VPS / Railway	cPanel compartido
Mejor para	Desarrolladores JS	Proyectos Arduino	Hosting clasico

Endpoints REST comunes a los tres backends:

Método	Endpoint	Descripción
POST	/api/readings	Recibe temperatura y humedad desde el script serial
GET	/api/readings?camera_id=1	Historial de lecturas de una cámara
GET	/api/cameras/status	Estado actual de todas las cámaras
GET	/api/readings/alerts	Lecturas fuera del rango 0 °C – 10 °C

4a. Node.js + Express

```
npm install express better-sqlite3 socket.io cors
```

Estructura de archivos:

```
backend-node/
  ├── server.js ← punto de entrada, configura Express + Socket.io
  ├── db.js ← conexión SQLite y esquema de tabla
  ├── routes/
  │   ├── readings.js ← endpoints /api/readings
  │   └── package.json
```

server.js (fragmento clave):

```
const express = require('express');
const { Server } = require('socket.io');
const db = require('./db');

const app = express();
app.use(express.json());

app.post('/api/readings', (req, res) => {
  const { camera_id, temperature, humidity } = req.body;
  const out_of_range = temperature < 0 || temperature > 10;
  db.prepare(
    'INSERT INTO readings VALUES (?, ?, ?, ?, CURRENT_TIMESTAMP)'
  ).run(camera_id, temperature, humidity, out_of_range ? 1 : 0);
  if (out_of_range) io.emit('alert', { camera_id, temperature });
  res.json({ ok: true });
});
```

```
app.listen(3000);
```

4b. Python + FastAPI

```
pip install fastapi uvicorn sqlite3
```

Estructura de archivos:

```
backend-python/  
■■■ main.py ← servidor FastAPI con todos los endpoints  
■■■ database.py ← conexión SQLite y helpers  
■■■ models.py ← esquemas Pydantic (validación automática)  
■■■ requirements.txt
```

main.py (fragmento clave):

```
from fastapi import FastAPI  
from pydantic import BaseModel  
import sqlite3, datetime  
  
app = FastAPI()  
  
class Reading(BaseModel):  
    camera_id: int  
    temperature: float  
    humidity: float  
  
@app.post('/api/readings')  
def post_reading(r: Reading):  
    out_of_range = not (0 <= r.temperature <= 10)  
    # guardar en SQLite...  
    if out_of_range:  
        # disparar alerta  
        pass  
    return {'ok': True}  
  
# Correr con: uvicorn main:app --reload  
# Docs en: http://localhost:8000/docs
```

Ventaja exclusiva de FastAPI: al correr el servidor, se genera automáticamente documentación interactiva en **<http://localhost:8000/docs>** donde podés probar todos los endpoints sin necesidad de Postman u otra herramienta.

4c. PHP + MySQL

Requiere: PHP 7.4+, MySQL/MariaDB, servidor Apache/Nginx o XAMPP

Estructura de archivos:

```
backend-php/  
  ■■■ api/  
  ■ ■■■ post_reading.php ← recibe datos del script serial  
  ■ ■■■ get_readings.php ← historial de una cámara  
  ■ ■■■ get_status.php ← estado actual de todas las cámaras  
  ■■■ config/  
    ■■■ db.php ← conexión MySQL
```

post_reading.php (fragmento clave):

```
<?php  
header('Content-Type: application/json');  
require '../config/db.php';  
  
$data = json_decode(file_get_contents('php://input'), true);  
$cam = (int)$data['camera_id'];  
$temp = (float)$data['temperature'];  
$hum = (float)$data['humidity'];  
$out = ($temp < 0 || $temp > 10) ? 1 : 0;  
  
$stmt = $pdo->prepare(  
    'INSERT INTO readings (camera_id, temperature, humidity, alert)  
    VALUES (?, ?, ?, ?)'  
);  
$stmt->execute([$cam, $temp, $hum, $out]);  
echo json_encode(['ok' => true]);  
?>
```

5. Capa 4 — Base de datos

El esquema es idéntico para SQLite, MySQL y PostgreSQL. Solo cambia la sintaxis menor de tipos de datos.

Esquema SQL:

```
CREATE TABLE cameras (  
    id INTEGER PRIMARY KEY,  
    name TEXT NOT NULL, -- ej: 'Camara 1 - Manzanas'  
    location TEXT,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE readings (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    camera_id INTEGER REFERENCES cameras(id),  
    temperature REAL NOT NULL, -- en grados Celsius  
    humidity REAL,  
    alert INTEGER DEFAULT 0, -- 1 si esta fuera de rango  
    recorded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
);
```

6. Orden de implementación recomendado

Paso	Tarea	Tecnología
1	Conectar DHT11 al Arduino y subir el sketch	Arduino IDE
2	Verificar datos en el monitor serial	Arduino IDE
3	Correr el script lector serial y ver logs en consola	Python + pyserial
4	Levantar el backend elegido (Node/Python/PHP)	Stack seleccionado
5	Conectar el script serial al backend	HTTP POST
6	Crear el dashboard web con gráficos en tiempo real	HTML + Chart.js
7	Agregar alertas visuales cuando la temp sale del rango	Frontend + WebSocket

Para continuar: pedile a Claude que genere el código completo de cualquiera de los tres backends, el script serial, o el dashboard web con gráficos en tiempo real.